

RAPPORT DE PROJET DE Fin d'ÉTUDES

NoExcuse

Application mobile de gestion personnelle intelligente



Réalisé par : Issam Bouffa

Encadré par : Mr. Mohammed Bousmah

Filière : 4ème Année Génie Informatique

Année universitaire : 2025–2026

May 21, 2026

Dédicaces

Je dédie ce travail à mes parents, qui ont toujours été une source de soutien inconditionnel, de patience et de confiance tout au long de mon parcours universitaire. Vos sacrifices, vos encouragements et votre foi en mes capacités m'ont permis de surmonter les obstacles et d'atteindre mes objectifs.

À mes frères, sœurs et amis, pour leur présence constante, leur écoute et leur soutien moral pendant les moments de doute.

À tous ceux qui s'efforcent d'améliorer leur discipline personnelle et de transformer leurs habitudes quotidiennes. Ce projet est le fruit d'efforts continus, d'apprentissages face aux erreurs techniques et de persévérance face aux défis logiques. Puisse ce travail inspirer l'organisation et la rigueur.

Remerciements

La réalisation de ce projet de fin d'études a été rendue possible grâce au soutien, aux conseils et à la collaboration de nombreuses personnes.

Je tiens tout d'abord à exprimer ma profonde gratitude à mon encadrant pour sa disponibilité, sa patience et la pertinence de ses remarques tout au long du développement de ce projet. Ses orientations méthodologiques et ses conseils techniques m'ont grandement aidé à structurer mon travail et à surmonter les difficultés liées à la mise en œuvre de l'architecture logicielle.

Mes sincères remerciements s'adressent également aux membres du corps enseignant de mon établissement pour la qualité de la formation dispensée, qui m'a fourni les bases conceptuelles indispensables à la réalisation de ce projet.

Enfin, je remercie chaleureusement les bêta-testeurs de l'application NoExcuse pour leurs précieux retours concernant l'ergonomie, la fluidité des écrans et la logique fonctionnelle du système d'alarme, ainsi que mes proches pour leur soutien moral indéfectible.

Résumé

Le projet **NoExcuse** consiste à concevoir et développer une application mobile Android native de productivité et de gestion personnelle intelligente. L'objectif principal est de fournir un cadre intégré aidant l'utilisateur à structurer son emploi du temps, planifier ses sessions d'étude, organiser ses entraînements sportifs et réguler ses cycles de sommeil grâce à des mécanismes contraignants et un accompagnement dynamique.

L'originalité du projet réside dans son architecture découplée et l'intégration d'un assistant virtuel propulsé par une intelligence artificielle générative (via l'API Groq). Contrairement aux solutions traditionnelles qui proposent des réponses statiques ou déconnectées du quotidien de l'utilisateur, NoExcuse extrait localement les données de la journée (tâches accomplies, heures d'étude programmées, exercices sportifs planifiés, régularité du sommeil) pour les injecter sous forme de contexte enrichi au modèle de langage. L'assistant virtuel devient ainsi un véritable coach de vie capable de fournir des évaluations contextuelles, réalistes et adaptées au profil de l'utilisateur.

Pour renforcer la discipline personnelle, l'application intègre également un système d'alarme strict dont la désactivation requiert le scan d'un code QR physique préalablement enregistré par l'utilisateur. Côté technique, l'application Android est développée nativement en Java, exploitant Room (SQLite) pour la persistance locale déconnectée et Retrofit pour la communication réseau. Le backend, développé avec le framework asynchrone FastAPI en Python, gère l'orchestration des données, la persistance centralisée avec MongoDB, et la liaison hautement optimisée avec le service IA.

Mots-clés : Android, Java, FastAPI, Room, SQLite, Intelligence Artificielle, MongoDB, Productivité, Discipline Personnelle.

Abstract

The **NoExcuse** project delivers a native Android mobile application focused on high-performance productivity and behavioral discipline management. The application aims to unify standard productivity resources by enabling users to seamlessly organize tasks, schedule targeted study blocks, track physical training sessions, and enforce sleep routine habits through binding in-app mechanisms.

A key contribution of this project is its decoupled architecture and the inclusion of a contextual AI-driven virtual assistant powered by the Groq API. While contemporary solutions offer generic advice, NoExcuse aggregates local, real-time metrics (e.g., daily task statuses, target study achievements, scheduled workouts, sleep consistency data) and projects them as prompt metadata to the Large Language Model. The virtual assistant functions as a highly personalized digital coach, providing motivational analysis and actionable insights directly relevant to the user's ongoing physical and mental state.

To promote active accountability, the application implements a wake-up alarm module requiring a physical QR code scan to turn off, preventing early morning procrastination. On the technical side, the native Android application is built in Java, leveraging the Room persistence library (SQLite) for offline-first capabilities and Retrofit for RESTful networking. The backend architecture utilizes Python's asynchronous FastAPI framework to handle application logic, data synchronization, persistent storage via MongoDB, and secure low-latency communication with the AI service.

Keywords: Android, Java, FastAPI, Room, SQLite, Artificial Intelligence, Productivity.

Liste des Abréviations

Abréviation	Signification
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AVD	Android Virtual Device
DAO	Data Access Object
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
LLM	Large Language Model
MVC	Model-View-Controller
MVVM	Model-View-ViewModel
NoSQL	Not Only SQL
ORM	Object-Relational Mapping
REST	Representational State Transfer
Room	Bibliothèque Android d'abstraction SQLite
SDK	Software Development Kit
SGBD	Système de Gestion de Base de Données
SQLite	Système de gestion de base de données de type embarqué
UML	Unified Modeling Language
UI	User Interface
UX	User Experience
XML	Extensible Markup Language

Table des figures

1.1	Chronogramme de planification et répartition des tâches (Gantt)	7
2.1	Diagramme UML de cas d'utilisation du système NoExcuse	10
2.2	Flux d'activité lors d'une requête d'assistance IA contextuelle	11
3.1	Architecture technique globale de l'écosystème NoExcuse	12
3.2	Diagramme de classes du système Android	13
4.1	Diagramme de séquence de la requête d'assistance IA	15
A.1	Écran principal de l'application NoExcuse	24
A.2	Menu de navigation latéral (Navigation Drawer)	25
A.3	Boîte de dialogue d'ajout de tâche quotidienne	25
A.4	Interface de planification d'une session d'étude	26
A.5	Interface de planification hebdomadaire de sport	26
A.6	Interface de discussion de l'assistant AI General	27
A.7	Avatar de l'assistant virtuel IA	27
A.8	Affichage Logcat de la charge JSON envoyée au backend	28
A.9	Logs d'exécution du serveur d'API REST FastAPI	28
A.10	Rapport de build réussi de la console Gradle	29

Liste des tableaux

1.1	Tableau comparatif des solutions de productivité du marché.....	5
3.1	Technologies utilisées et justifications techniques.....	14
5.1	Cahier de tests et de validation du système NoExcuse.....	19

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des Abréviations	v
Liste des Figures	vi
Liste des Tableaux	vii
Introduction Générale	1
1 Contexte Général du Projet	3
Introduction.....	3
1.1 Présentation du Projet.....	3
1.1.1 Problématique.....	3
1.1.2 Étude de l’Existant.....	4
1.1.3 Solution Proposée.....	5
1.2 Méthodologie de Travail et Planification	6
1.2.1 Découpage en Sprints.....	6
1.2.2 Planification Temporelle (Gantt)	6
Conclusion.....	7
2 Analyse et Conception	8
Introduction.....	8
2.1 Étude Fonctionnelle et Besoins	8
2.1.1 Analyse des Besoins Fonctionnels.....	8

2.1.2	Analyse des Besoins Non Fonctionnels.....	9
2.2	Etude Conceptuelle et Règles de Gestion.....	9
2.3	Modélisation UML.....	10
2.3.1	Diagramme de Cas d'Utilisation.....	10
2.3.2	Diagramme d'Activité - Chat IA Contextuel.....	10
	Conclusion.....	11
3	Etude Technique	12
	Introduction.....	12
3.1	Architecture Globale du Système.....	12
3.2	Conception de la Structure des Classes	13
3.3	Technologies Utilisées.....	13
	Conclusion.....	14
4	Mise en œuvre	15
	Introduction.....	15
4.1	Déroulement des Interactions IA (Séquence).....	15
4.2	Réalisation des Modules Android.....	16
4.2.1	Module Gestion des Tâches.....	16
4.2.2	Module Éducation et Sessions d'Étude	16
4.2.3	Module d'Entraînement Physique (Gym).....	16
4.2.4	Module Sommeil et Alarme Intelligente.....	16
4.2.5	L'Assistant Virtuel IA	16
4.3	Pratiques d'Intégration et DevOps.....	17
	Conclusion.....	17
5	Tests et Validation	18
	Introduction.....	18
5.1	Stratégie de Test Globale.....	18
5.2	Cahier de Recette Fonctionnel.....	18
5.3	Analyse des Résultats	19
	Conclusion.....	20
	Conclusion Générale et Perspectives	21
	Références	23
A	Annexes	24
A.1	Captures d'écran et Interfaces de l'Application.....	24

Introduction Générale

Dans notre société moderne caractérisée par un flux constant d'informations et une sollicitation permanente de notre attention par les technologies numériques, le maintien d'une hygiène de vie saine et d'une productivité rigoureuse est devenu un défi quotidien. Les distractions sont omniprésentes, et le phénomène de procrastination touche une large part de la population active et étudiante. Pour lutter contre ce manque d'organisation personnelle, de nombreuses applications mobiles ont vu le jour.

Toutefois, la majorité des outils actuels souffrent d'une forte fragmentation fonctionnelle. L'utilisateur se retrouve contraint d'utiliser une application pour gérer ses listes de tâches, une autre pour suivre ses séances de musculation, un réveil tiers pour s'assurer d'éviter les pannes de réveil, et des carnets physiques ou numériques pour planifier ses sessions d'étude. De surcroît, ces applications agissent comme des réceptacles passifs d'informations : elles n'offrent aucune rétroaction dynamique, aucune analyse intelligente et aucun soutien de motivation tenant compte de la réalité globale de la journée de l'utilisateur.

Le projet **NoExcuse** émerge de la volonté de résoudre cette fragmentation en proposant une plateforme mobile native Android unifiée, intelligente et proactive. L'application intègre au sein d'une même interface les quatre piliers fondamentaux de la discipline quotidienne :

1. **La gestion du temps et des tâches** : un suivi des priorités de la journée.
2. **L'apprentissage intellectuel** : une planification ciblée des sessions d'étude.
3. **Le maintien de la forme physique** : un planificateur hebdomadaire d'entraînements sportifs.
4. **La régulation biologique** : un système de réveil strict reposant sur la contrainte physique du scan de code QR.

Le cœur innovant de NoExcuse repose sur l'exploitation des données locales collectées de manière éthique à travers ces quatre modules. En croisant ces informations et en

Rapport de Projet de Fin d'Études

les transmettant de manière sécurisée à un backend asynchrone FastAPI connecté à des modèles de traitement automatique du langage naturel via l'API Groq, l'application transcende le rôle de simple outil passif. Elle propose un assistant virtuel intelligent capable de formuler des conseils personnalisés, de réajuster les objectifs de l'utilisateur en fonction de sa fatigue ou de ses retards d'étude, et d'agir comme un véritable coach personnel.

Ce rapport retrace de manière détaillée la genèse, l'analyse, la conception, l'architecture technique et l'implémentation de la solution NoExcuse. Le premier chapitre définit le contexte général, l'étude du besoin ainsi que la planification du projet. Le deuxième chapitre est consacré à l'analyse fonctionnelle et conceptuelle à travers les diagrammes d'utilisation et d'activité. Le troisième chapitre présente l'étude technique comparative des choix technologiques et de l'architecture logicielle MVVM. Le quatrième chapitre expose de manière concrète la mise en œuvre, illustrée par des extraits de code structurants. Enfin, le cinquième chapitre détaille la phase de tests et de validation, avant de clore ce rapport par une conclusion générale ouvrant sur les perspectives de ce travail.

Contexte Général du Projet

Introduction

Ce premier chapitre pose les bases de notre réflexion en présentant le cadre de l'application NoExcuse. Nous y étudions la problématique de la discipline personnelle, analysons l'existant sur le marché, décrivons la solution conceptuelle proposée et explicitons la méthodologie agile adoptée tout au long du cycle de développement.

1.1 Présentation du Projet

L'application NoExcuse est née du constat que l'autodiscipline ne dépend pas uniquement de la volonté individuelle, mais est fortement influencée par l'environnement et l'accès à des outils d'organisation fluides. NoExcuse se positionne comme un système d'exploitation de la vie quotidienne de l'utilisateur, visant à maximiser son efficacité tout en réduisant la charge cognitive nécessaire à la planification.

1.1.1 Problématique

L'abondance d'applications mobiles de productivité n'a pas résolu la problématique de la dispersion de l'attention. Les limites des solutions existantes s'articulent autour de trois axes :

- **La fragmentation fonctionnelle :** Les données de sommeil (provenant de trackers d'activité), les données d'entraînement (provenant d'applications de fitness) et les listes de tâches (provenant d'outils de Kanban) ne se croisent jamais. Il est donc impossible pour un utilisateur de constater de manière analytique qu'un manque de sommeil de deux jours affecte directement la réalisation de ses sessions d'étude ou la performance de ses séances de sport.

- **L'absence de contrainte physique (Friction active) :** La plupart des alarmes de réveil peuvent être désactivées d'un simple glissement de doigt, favorisant le phénomène de *snoozing* qui perturbe les cycles de sommeil et cause des retards dès le début de la journée.
- **L'inefficacité des assistants virtuels génériques :** Les assistants virtuels actuels ne connaissent rien de la vie réelle de l'utilisateur au moment où celui-ci interagit avec eux. Demander un conseil de gestion du temps à une IA classique nécessite de rédiger un long paragraphe de contexte. L'utilisateur abandonne souvent cette démarche en raison de la lourdeur du processus.

La question centrale de notre projet s'énonce ainsi : *Comment concevoir et développer un système logiciel unifié capable de collecter de manière sécurisée le contexte quotidien d'un utilisateur afin d'alimenter un système d'assistance cognitive réactif tout en imposant des verrous physiques favorisant la discipline ?*

1.1.2 Étude de l'Existant

Afin de positionner l'application NoExcuse sur le marché des logiciels d'organisation, nous avons mené une étude comparative de plusieurs outils populaires.

TABLE 1.1 : Tableau comparatif des solutions de productivité du marché

Solution	Points Forts	Points Faibles	Alternative NoExcuse
Todoist	Ergonomie, gestion de projet, rappels rapides.	Payant pour les fonctions avancées, aucune dimension bien-être ou sport.	Intégration complète avec les modules d'étude et de sport.
Habitica	Gamification intéressante sous forme de RPG.	Graphismes rétro clivants, lassitude rapide des utilisateurs non-joueurs.	Coaching IA réel au lieu de simples récompenses virtuelles.
Alarmy	Missions de réveil complexes (calculs, secousses).	Publicités intrusives, aucune de listes de tâches.	Réveil par QR code gratuit et intégré aux modules quotidiens.
ChatGPT Mobile	Grande culture générale, rédaction rapide.	Aucune idée du contexte physique réel de l'utilisateur.	Prompt contextuel dynamique s'appuyant sur Room/FastAPI.

1.1.3 Solution Proposée

NoExcuse propose de lever ces verrous grâce à une architecture logicielle articulée autour de deux composants majeurs : une application Android native de haute performance et un serveur API asynchrone chargé du traitement intelligent des requêtes.

- **L'approche "Offline-First" :** L'application garantit que toutes les actions quotidiennes (création de tâches, validation de sport, programmation d'alarmes) fonctionnent sans connexion réseau grâce à la base de données SQLite/Room locale.
- **L'approche "Context-Aware" (Sensible au contexte) :** Lors de l'envoi d'un message à l'assistant, l'application extrait automatiquement l'état courant de l'appareil (ex : "Tâche 1 : Terminée", "Session d'étude Mathématiques : En retard", "Heure de sommeil cible : 23h00, Réveillé à 07h00") et fusionne cet état dans la requête de l'utilisateur pour formuler un conseil immédiatement applicable.

- **Le verrou anti-procrastination** : Le module de réveil force le réveil effectif de l'utilisateur en coupant la sonnerie uniquement lorsque la caméra de l'appareil détecte la signature géométrique d'un QR code spécifique.

1.2 Méthodologie de Travail et Planification

La réalisation du projet NoExcuse a suivi la méthodologie Agile Scrum, particulièrement adaptée aux projets de développement logiciel à cycle rapide.

1.2.1 Découpage en Sprints

Le projet a été segmenté en quatre sprints majeurs de deux semaines chacun :

- **Sprint 1 (Analyse & Spécifications)** : Rédaction du cahier des charges, modélisation UML, configuration des environnements (Android Studio, FastAPI, MongoDB) et initialisation du dépôt de code Git.
- **Sprint 2 (Développement du Noyau Android)** : Implémentation de la persistance locale Room, création des entités logicielles, développement des écrans de gestion des tâches, de planification d'études et de sport.
- **Sprint 3 (Module d'Alarme et Backend FastAPI)** : Intégration de l'API CameraX pour le scan de QR codes, mise en place des services d'alarmes persistants, écriture des routes API FastAPI et interconnexion avec MongoDB.
- **Sprint 4 (Intégration de l'IA, Tests & Optimisation)** : Liaison avec l'API Groq, test de prompt engineering, réalisation des plans de tests unitaires et de charge, ajustement de l'interface graphique (UI/UX) et finalisation de la documentation technique.

1.2.2 Planification Temporelle (Gantt)

La planification globale a permis de respecter rigoureusement les échéances fixées. La figure 1.1 présente le planning de Gantt de l'ensemble du projet, détaillant l'imbrication des phases de test, de codage et d'intégration afin de livrer les fonctionnalités de manière incrémentale.

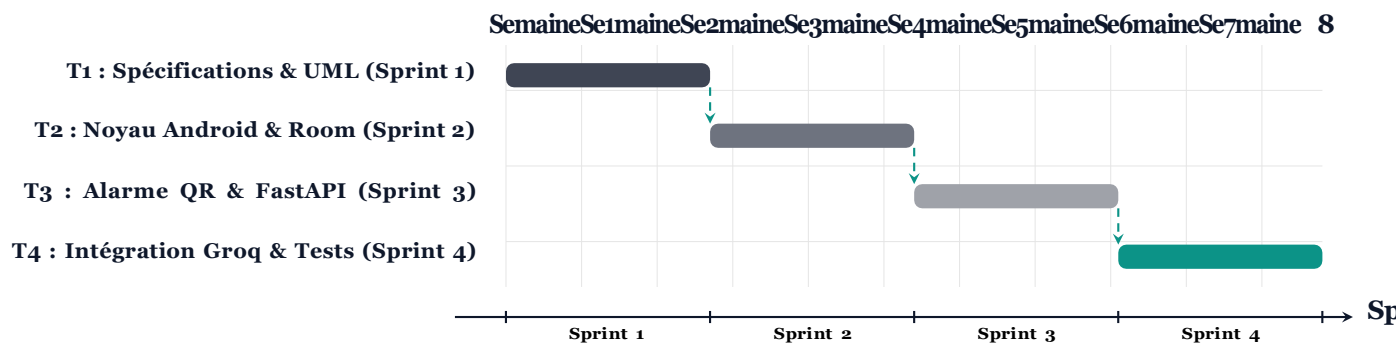


FIGURE 1.1 : Chronogramme de planification et répartition des tâches (Gantt)

Conclusion

Ce chapitre a mis en lumière le cadre de développement de NoExcuse et la problématique de fragmentation applicative qu'elle vise à résoudre. Le chapitre suivant détaille l'analyse fonctionnelle et conceptuelle du système à l'aide de cas d'utilisation et de scénarios d'activité.

Analyse et Conception

Introduction

La phase d'analyse et de conception est indispensable pour traduire les besoins des utilisateurs en spécifications techniques claires et exploitables. Ce chapitre présente les exigences fonctionnelles et non fonctionnelles du système, définit les règles de gestion métier et expose la conception UML globale de l'écosystème NoExcuse.

2.1 Étude Fonctionnelle et Besoins

2.1.1 Analyse des Besoins Fonctionnels

Les besoins fonctionnels identifiés pour l'application NoExcuse décrivent l'éventail d'actions réalisables par l'utilisateur final :

- Créer, consulter, modifier et supprimer des tâches quotidiennes.
- Marquer une tâche quotidienne comme terminée.
- Configurer une session d'étude (choix du module, plan d'apprentissage, horaires).
- Établir un programme de musculation ou de cardio hebdomadaire avec liste d'exercices.
- Définir les paramètres d'alarme et de suivi du sommeil.
- Choisir le niveau de contrainte de l'alarme (activation du scan de QR code).
- Consulter un tableau de bord récapitulant les activités planifiées de la journée.
- Envoyer des messages de discussion à l'assistant virtuel et recevoir des réponses contextualisées.

2.1.2 Analyse des Besoins Non Fonctionnels

Les besoins non fonctionnels sont essentiels pour garantir une expérience utilisateur fluide, performante et sécurisée :

- **Ergonomie** : interface simple, sombre et adaptée à une utilisation quotidienne, réduisant la fatigue oculaire.
- **Performance** : accès rapide aux données locales grâce à SQLite/Room (temps d'accès inférieur à 100ms).
- **Fiabilité** : conservation des données locales même hors connexion, avec un fonctionnement complet du système d'alarme.
- **Maintenabilité** : structuration stricte du code en couches distinctes (UI, Repository, DAO, API).
- **Sécurité** : gestion des secrets côté serveur (clé API Groq non embarquée dans l'APK).

2.2 Etude Conceptuelle et Règles de Gestion

Les règles de gestion régissent l'interaction des entités logicielles au sein de l'application :

1. Une tâche quotidienne possède un titre, une description, une heure et un état de réalisation.
2. Une session d'étude possède un module, un plan, une heure de début, une heure de fin et un état.
3. Un plan de sport est associé à un jour de la semaine et à une semaine donnée, et contient plusieurs exercices.
4. L'assistant IA doit recevoir les données du jour (tâches, sport, sommeil) avant de générer une réponse personnalisée.
5. Les paramètres de sommeil les plus récents sont appliqués par défaut comme contexte courant.

2.3 Modélisation UML

La conception du système a été modélisée à l'aide de différents diagrammes UML décrivant les cas d'utilisation, l'activité de communication de l'IA, la structure des classes et l'architecture globale.

2.3.1 Diagramme de Cas d'Utilisation

Le diagramme de la figure 2.1 synthétise l'ensemble des cas d'utilisation gérés par l'application NoExcuse. L'utilisateur interagit directement avec les modules locaux de planification et de suivi tandis que le backend fait office de relais avec les services d'IA.

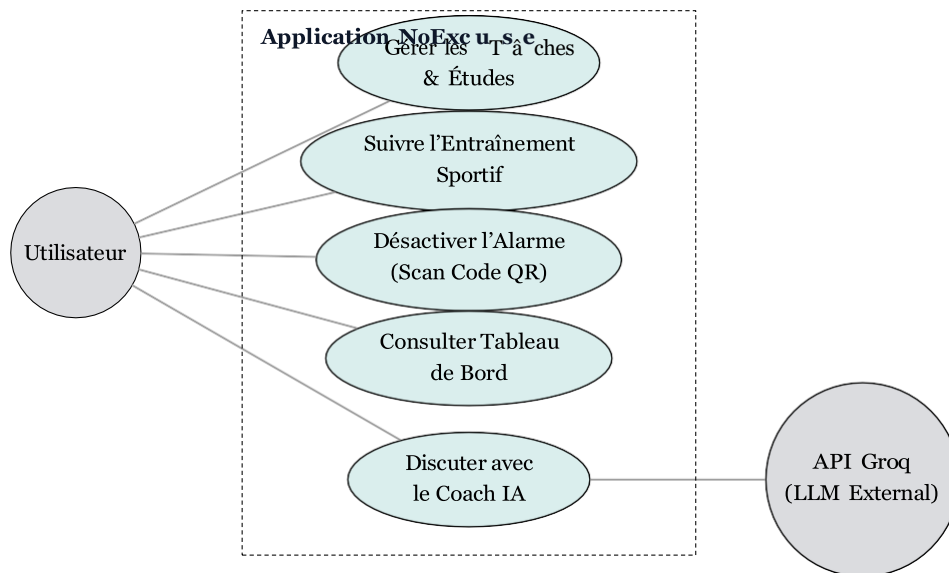


FIGURE 2.1 : Diagramme UML de cas d'utilisation du système NoExcuse

2.3.2 Diagramme d'Activité - Chat IA Contextuel

Le diagramme d'activité présenté à la figure 2.2 montre le flux d'exécution et les prises de décisions logiques lors de l'envoi d'un message à l'assistant IA. Le système extrait les données locales de Room avant de composer le corps de la requête.

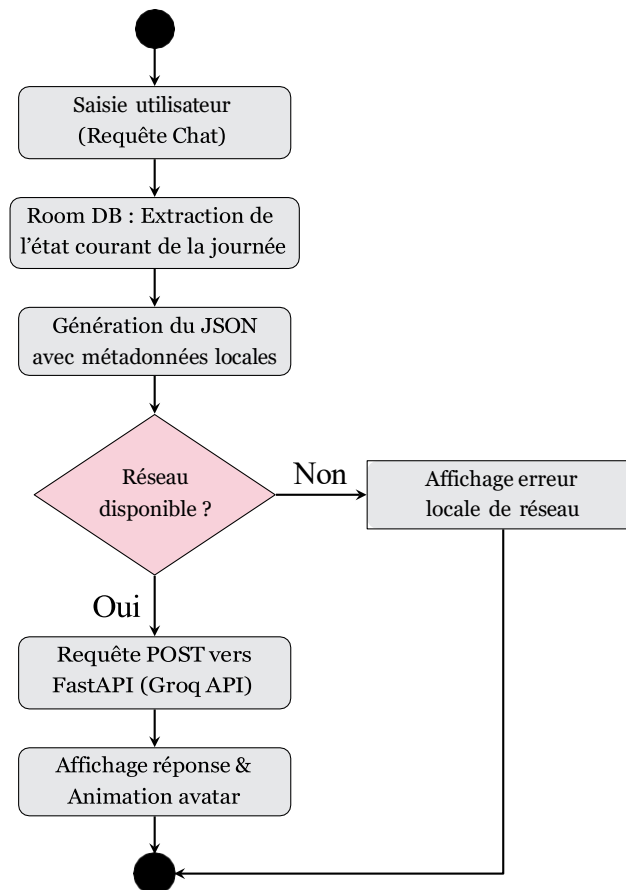


FIGURE 2.2 : Flux d'activité lors d'une requête d'assistance IA contextuelle

Conclusion

Ce chapitre a permis de structurer les besoins fonctionnels et de présenter les diagrammes UML préliminaires. Le chapitre suivant détaille l'étude technique comparative des choix d'outils et de l'architecture logicielle MVVM de l'application.

Etude Technique

Introduction

Ce chapitre présente l'architecture technique du projet, les choix technologiques et l'environnement de travail. Nous détaillons la modélisation des classes logicielles ainsi que le schéma global d'architecture système.

3.1 Architecture Globale du Système

L'architecture se compose d'une application Android native (client) et d'un backend REST FastAPI (serveur) assurant le relai vers MongoDB et Groq AI. La figure 3.1 illustre cette séparation claire des responsabilités :

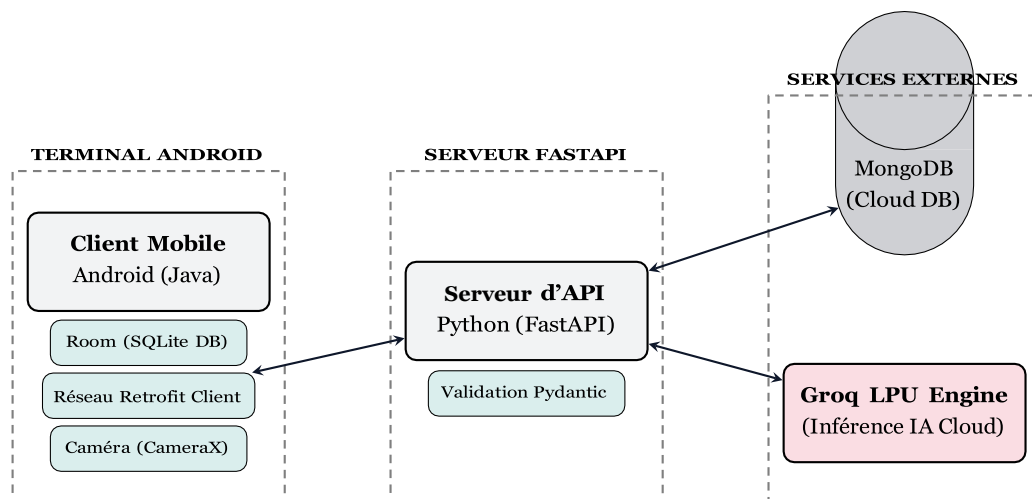


FIGURE 3.1 : Architecture technique globale de l'écosystème NoExcuse

3.2 Conception de la Structure des Classes

La figure 3.2 présente le diagramme de classes simplifié de l'application mobile, montrant les entités persistées sous Room (SQLite) et l'interface de communication avec le serveur REST.

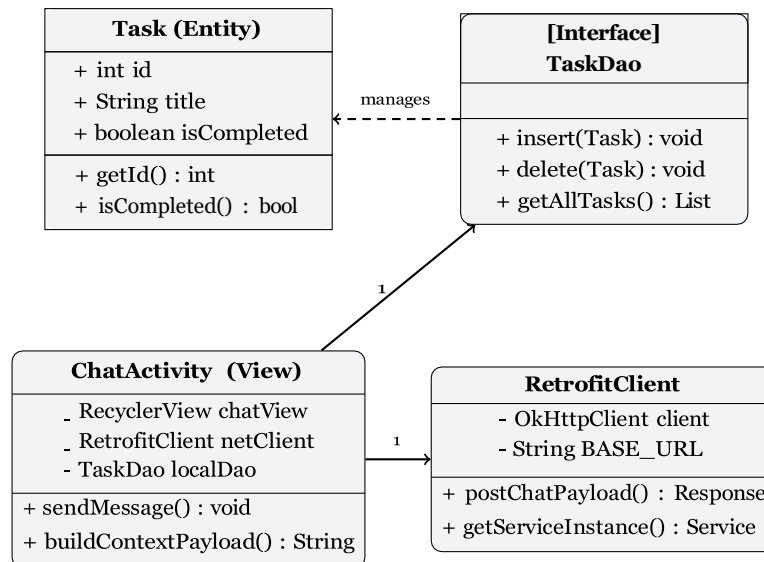


FIGURE 3.2 : Diagramme de classes du système Android

3.3 Technologies Utilisées

L'application s'appuie sur une pile technologique moderne détaillée dans le tableau 3.1 :

TABLE 3.1 : Technologies utilisées et justifications techniques

Technologie	Rôle et justification technique
Java	Langage natif Android assurant robustesse et compatibilité avec les composants d'arrière-plan du système.
Room (SQLite)	Abstraction d'accès à SQLite pour assurer la persistance locale déconnectée et la réactivité de l'application.
Retrofit	Client HTTP de type-safe facilitant l'interfaçage avec le backend de l'API REST.
FastAPI	Framework asynchrone Python choisi pour sa vitesse d'exécution et sa validation automatique de données par Pydantic.
MongoDB	Base de données NoSQL stockant de manière flexible l'historique de chat et les catalogues d'exercices.
Groq API	Service d'inférence LLM à ultra-basse latence assurant un temps de réponse rapide de l'assistant virtuel.

Conclusion

L'étude technique confirme le choix d'une architecture client-serveur découplée et robuste. Le chapitre suivant présente l'implémentation de ces différents modules au sein de l'application et du serveur FastAPI.

Mise en œuvre

Introduction

Ce chapitre présente la mise en œuvre pratique de l'application NoExcuse, en abordant la réalisation de chaque module, l'échange de données avec le backend FastAPI et les configurations DevOps associées.

4.1 Déroulement des Interactions IA (Séquence)

Le diagramme de la figure 4.1 détaille le flux chronologique des messages échangés entre l'utilisateur, l'interface de chat Android, la base de données Room locale, le serveur d'API FastAPI et le moteur d'IA de Groq.

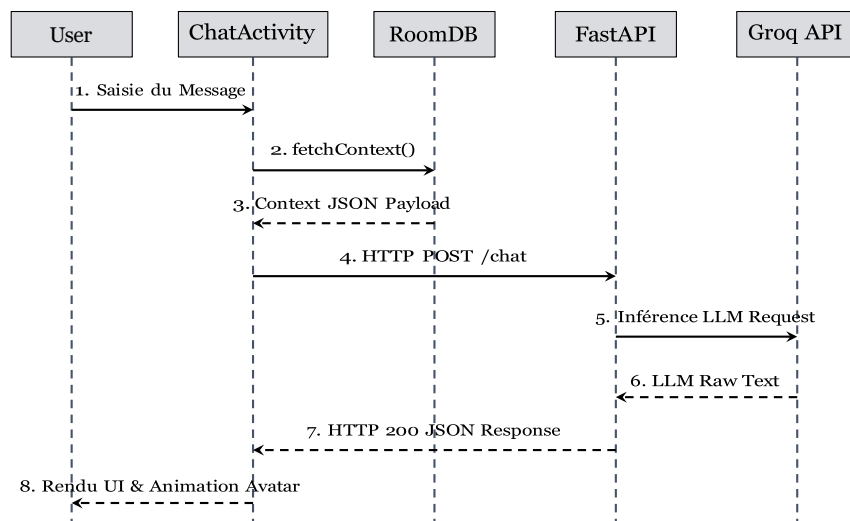


FIGURE 4.1 : Diagramme de séquence de la requête d'assistance IA

4.2 Réalisation des Modules Android

4.2.1 Module Gestion des Tâches

Le module permet de stocker et d'interagir localement avec des instances de la table tasks de la base de données SQLite. L'utilisateur peut marquer chaque tâche comme terminée d'un simple clic sur la case à cocher correspondante.

4.2.2 Module Éducation et Sessions d'Étude

L'utilisateur planifie ses blocs horaires d'étude. Ces sessions, caractérisées par un module et un objectif spécifique, fournissent à l'assistant IA des informations précieuses pour analyser les progrès académiques.

4.2.3 Module d'Entraînement Physique (Gym)

Permet la planification de séances de musculation ou d'activités cardio hebdomadaires. Le module stocke la partie du corps ciblée et la liste d'exercices associés.

4.2.4 Module Sommeil et Alarme Intelligente

L'application force la vigilance matinale en déclenchant un signal sonore qui ne peut être éteint que par le scan de la signature numérique du code QR enregistré au préalable par l'utilisateur.

4.2.5 L'Assistant Virtuel IA

La compilation du contexte local de productivité permet de construire un prompt système directif, transformant l'agent conversationnel en un coach de vie ferme et réactif.

Listing 4.1 : Extraction locale des données Room pour l'appel d'API

```
1 body . put(" message ",      message );
2 body . put(" daily_tasks",    daily Tasks);
3 body.put(" education_tasks", educationTasks);
4 body . put(" gym_plans",      gym Plans);
5 body . put(" sleep_settings", sleep Settings);
```

4.3 Pratiques d'Intégration et DevOps

Le processus de construction et de déploiement de l'application s'appuie sur le moteur Gradle pour Android et le serveur ASGI Uvicorn pour FastAPI :

Listing 4.2 : Compilation de l'APK via Gradle

```
1 gradlew .bat assembleDebug
```

Le backend Python est initialisé sur le serveur de calcul grâce à la commande d'écoute réseau :

Listing 4.3 : Lancement du serveur de production FastAPI

```
1 uvicorn main:app --host 0.0.0.0 --port 8000
```

La clé API Groq est passée sous forme de variable d'environnement système afin d'éviter les risques de vol de jetons d'accès lors de l'hébergement du code sur GitHub.

Conclusion

La mise en œuvre a validé la synergie entre la persistance réactive locale (Room) et la souplesse algorithmique du backend (FastAPI, Groq). Le chapitre suivant détaille l'analyse fonctionnelle et technique à travers les plans de tests fonctionnels et d'intégration.

Tests et Validation

Introduction

La validation d'un système logiciel nécessite d'éprouver chaque module face à des scénarios limites et à des utilisations dégradées (erreurs réseau, entrées utilisateur incorrectes, etc.). Ce chapitre présente notre stratégie globale de tests, liste notre cahier de tests fonctionnels et analyse les résultats de validation obtenus.

5.1 Stratégie de Test Globale

Notre plan de validation s'est structuré autour de trois approches complémentaires :

- **Les Tests Unitaires** : Validation isolée de la logique métier (calcul d'intervalles de temps de sommeil, validation de la chaîne de caractères d'un QR code décodé, conversion de dates).
- **Les Tests d'Intégration** : Validation de la bonne transmission des données JSON entre l'application cliente Android et le backend FastAPI (contrôle de la cohérence des schémas Pydantic).
- **Les Tests de Robustesse (Edge Cases)** : Validation du comportement de l'alarme en mode hors ligne, et gestion des échecs d'appels à l'API de traitement du langage naturel Groq.

5.2 Cahier de Recette Fonctionnel

Pour nous assurer de la conformité de l'application NoExcuse par rapport aux spécifications détaillées dans le chapitre 2, nous avons exécuté le plan de tests structuré présenté dans le tableau 5.1 :

TABLE 5.1 : Cahier de tests et de validation du système NoExcuse

ID Test	Scénario descriptif	Résultat attendu	Statut	Observations
T-001	Création d'une tâche valide sans description.	Tâche créée avec le titre saisi. La description affiche un message par défaut.	Conforme	Enregistré avec succès dans Room.
T-002	Déclenchement de l'alarme avec l'option QR code activée.	L'alarme retentit. Le bouton de coupure classique est invisible. Seul le scan de QR code est disponible.	Conforme	La sonnerie est persistante même en cas de fermeture forcée de l'interface.
T-003	Scan d'un code QR invalide pendant l'alarme.	L'alarme continue de sonner, un message d'erreur indique que le code scanné n'est pas le bon.	Conforme	L'application maintient l'état d'alerte.
T-004	Envoi d'une requête IA sans connexion réseau sur le téléphone.	L'application détecte la perte de réseau et affiche immédiatement un message d'invitation à se reconnecter.	Conforme	Empêche l'application de bloquer en attente d'un timeout de connexion.
T-005	Envoi d'un ensemble de données vide au backend FastAPI.	Le backend retourne une erreur HTTP 422 avec les détails des champs manquants.	Conforme	La validation par Pydantic fonctionne de manière nominale.

5.3 Analyse des Résultats

Les tests réalisés confirment la robustesse de l'application NoExcuse :

- **Intégrité des données** : Le recours à la bibliothèque Room prévient tout cas de

corruption de la base de données SQLite locale, même lors de l'arrêt inattendu de l'appareil.

- **Latence de l'IA :** Les mesures d'intégration réseau révèlent un temps de réponse moyen de l'assistant IA compris entre 1,2 et 2,8 secondes lors d'une connexion 4G ou Wi-Fi standard. Ce résultat s'explique par la très faible latence d'inférence proposée par les serveurs de calcul de Groq.
- **Ressources système d'Android :** L'utilisation de la caméra arrière via l'API CameraX de Google pour l'analyse en temps réel du code QR consomme de l'énergie de manière ponctuelle et ne provoque aucun échauffement de l'appareil, l'analyse étant coupée dès l'extinction validée de l'alarme.

Conclusion

La phase de validation a permis d'éprouver l'ensemble de notre architecture matérielle et logicielle. NoExcuse se comporte de manière stable lors de tous les scénarios critiques identifiés.

Conclusion Générale et Perspectives

La réalisation du projet **NoExcuse** a permis d'aboutir à la conception et à l'implémentation d'une application mobile Android native complète, performante et innovante, dédiée à l'autodiscipline et à la productivité personnelle. En combinant judicieusement les techniques classiques d'organisation du temps à des contraintes physiques et logiques, et en y adjoignant la puissance d'une intelligence artificielle générative contextualisée, NoExcuse se démarque des outils d'organisation traditionnels.

Sur le plan technique, l'implémentation de l'architecture logicielle client-serveur a mis en évidence l'intérêt de séparer rigoureusement les responsabilités. L'utilisation du patron de conception MVVM (Model-View-ViewModel) côté client Android garantit la maintenabilité du code source Java et une séparation propre de l'affichage. L'abstraction offerte par la bibliothèque Room a permis de concrétiser une approche de développement déconnectée performante. Du côté du serveur, l'adoption du framework asynchrone FastAPI couplé à MongoDB a offert une grande souplesse dans la manipulation de structures de données variables tout en assurant d'excellents temps de traitement.

Ce travail ouvre de nombreuses perspectives d'amélioration et d'enrichissement fonctionnel :

- **Amélioration esthétique de l'assistant** : Modéliser un avatar virtuel interactif animé en temps réel à l'aide de moteurs graphiques légers (par exemple, Lottie ou des rendus 2D vectoriels), dont les mouvements ou expressions faciales s'adaptent automatiquement aux réponses et conseils humoristiques ou fermes fournis par l'intelligence artificielle.
- **Synchronisation multcloud bidirectionnelle** : Mettre en place un système de réplication des données de la base Room locale vers l'instance de stockage MongoDB distante, afin de prémunir l'utilisateur de toute perte d'historique en cas de casse ou de changement de son terminal Android.
- **Sécurité et authentification renforcées** : Développer un module complet de gestion des utilisateurs reposant sur le protocole OAuth2 et le cryptage des données

Rapport de Projet de Fin d'Études

personnelles stockées, afin de garantir l'anonymat des informations de sommeil ou d'étude partagées avec les services tiers d'IA.

- **Intégration d'un tableau de bord de statistiques (Analytics) :** Générer des représentations graphiques (courbes, diagrammes en secteurs) montrant l'évolution du taux de réussite de l'utilisateur sur une base mensuelle (rapport entre les sessions d'études prévues et réalisées, régularité de l'heure de coucher).
- **Extension linguistique multilingue :** Adapter les invites de commande du système d'intelligence artificielle pour assurer un traitement bilingue/trilingue s'adaptant à l'usage de langues locales (Darija marocain, français, anglais).

En définitive, ce projet a représenté une opportunité enrichissante de mettre en pratique des compétences variées en développement mobile natif, en architecture d'API, en gestion de bases de données distribuées et en ingénierie de prompt, tout en proposant une réponse concrète à une problématique de gestion et d'organisation personnelle quotidienne.

Bibliographie

- [1] Google Developers, *Android Developers Documentation - Guides and API References*, disponible en ligne : <https://developer.android.com/>.
- [2] Google Developers, *Save data in a local database using Room*, Android Jetpack Guide, disponible en ligne : <https://developer.android.com/training/data-storage/room>.
- [3] Sebastián Ramírez, *FastAPI Documentation, High-performance web framework for building APIs with Python*, disponible en ligne : <https://fastapi.tiangolo.com/>.
- [4] Square Inc., *Retrofit - A type-safe HTTP client for Android and Java*, disponible en ligne : <https://square.github.io/retrofit/>.
- [5] MongoDB Inc., *The MongoDB Database Documentation - Flexible Document Model*, disponible en ligne : <https://www.mongodb.com/docs/>.
- [6] Groq Technologies, *Groq LPU System Inference Engine API Specifications*, disponible en ligne : <https://console.groq.com/docs>.
- [7] Samuel Colvin et al., *Pydantic - Data validation and settings management using Python type hints*, disponible en ligne : <https://docs.pydantic.dev/>.
- [8] Google Developers, *CameraX Jetpack Support Library Reference*, disponible en ligne : <https://developer.android.com/training/camerax>.

Annexes

A.1 Captures d'écran et Interfaces de l'Application

Cette section regroupe les figures illustrant les principales interfaces utilisateur de l'application mobile NoExcuse et l'affichage des logs système.

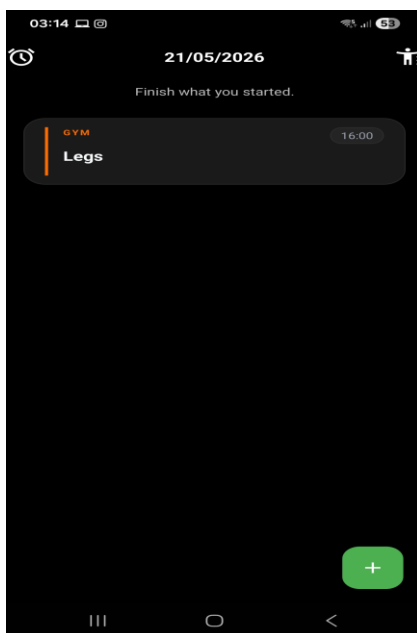


FIGURE A.1 : Écran principal de l'application NoExcuse

La figure A.1 présente l'interface d'accueil. L'utilisateur y trouve la date courante, une citation de motivation, et la liste dynamique de ses tâches de la journée récupérées depuis la base Room locale.

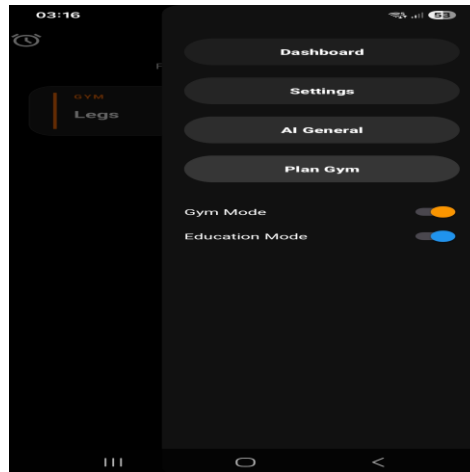


FIGURE A.2 : Menu de navigation latéral (Navigation Drawer)

La figure A.2 illustre le menu glissant qui permet de basculer rapidement entre le Dashboard, la page de discussion de l'assistant IA, le calendrier de sport et les paramètres généraux.

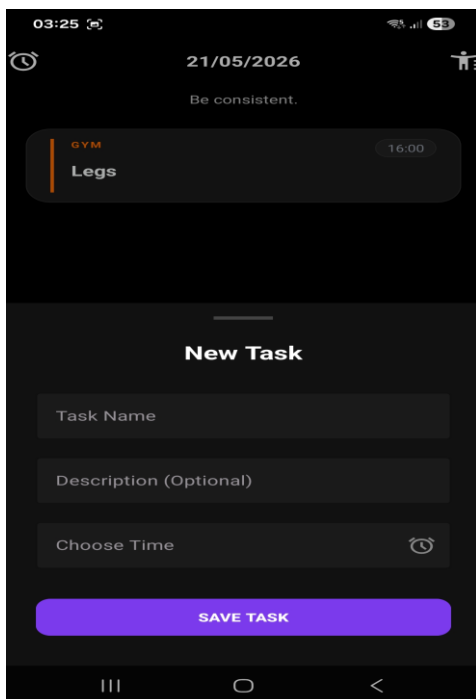
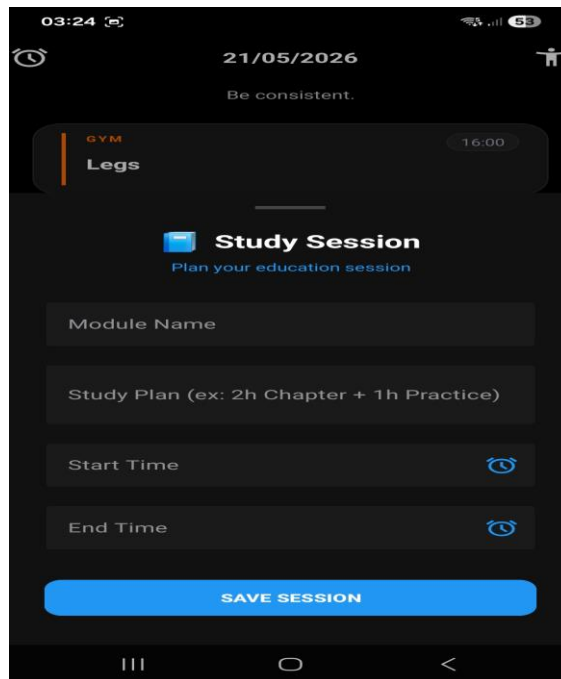


FIGURE A.3 : Boîte de dialogue d'ajout de tâche quotidienne

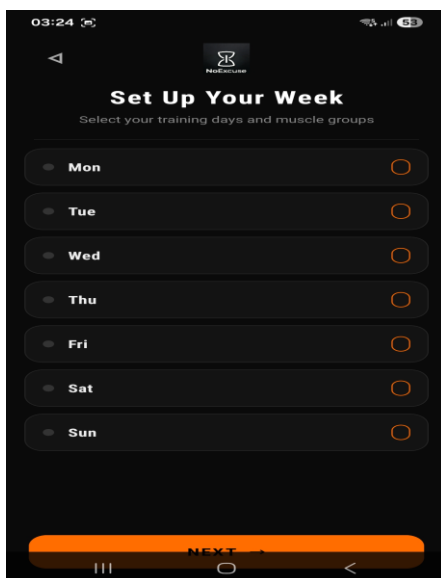
La figure A.3 montre la fenêtre surgissante de création de tâche. L'utilisateur peut y saisir un titre, une description libre et programmer l'heure prévue.



The screenshot shows a mobile app interface for planning a study session. At the top, the status bar displays the time 03:24, signal strength, and battery level at 53%. The app header includes a clock icon, the date 21/05/2026, and the motivational text 'Be consistent.'. Below this, a 'GYM' tab is active, showing 'Legs' and a duration of '16:00'. The main section is titled 'Study Session' with the subtitle 'Plan your education session'. It contains four input fields: 'Module Name', 'Study Plan (ex: 2h Chapter + 1h Practice)', 'Start Time' (with a clock icon), and 'End Time' (with a clock icon). A large blue button at the bottom is labeled 'SAVE SESSION'. The bottom navigation bar shows three icons: a list, a home icon, and a back arrow.

FIGURE A.4 : Interface de planification d'une session d'étude

La figure A.4 représente le formulaire de paramétrage d'étude. Les données de matière et d'objectifs y sont définies avant d'être envoyées au backend pour contextualiser l'IA.



The screenshot shows a mobile app interface for setting up a weekly sports schedule. The title is 'Set Up Your Week' with the subtitle 'Select your training days and muscle groups'. Below this, there is a list of days from Monday to Sunday, each with a radio button for selection. At the bottom, there is a large orange button labeled 'NEXT' and a back arrow icon. The bottom navigation bar shows three icons: a list, a home icon, and a back arrow.

FIGURE A.5 : Interface de planification hebdomadaire de sport

La figure A.5 montre l'écran de consultation des entraînements de la semaine, permettant d'affecter à chaque jour un groupe musculaire cible et d'en lister les exercices.



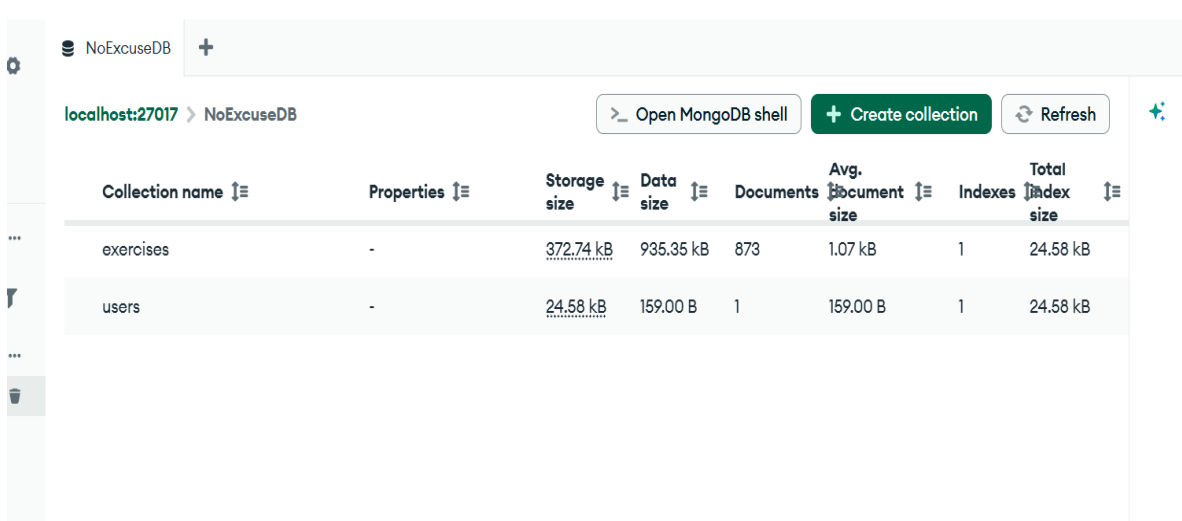
FIGURE A.6 : Interface de discussion de l'assistant AI General

La figure A.6 montre l'interface de conversation avec l'assistant. L'historique des bulles de messages reçus et envoyés s'y affiche de manière ergonomique.



FIGURE A.7 : Avatar de l'assistant virtuel IA

La figure A.7 montre l'avatar dynamique affiché en haut du chat de discussion NoExcuse. Ses différentes formes de bouche s'activent pour simuler la prononciation du texte généré par l'IA.



The screenshot shows the MongoDB NoExcuseDB interface. At the top, there's a header with 'NoExcuseDB' and a '+' icon. Below it, the address bar shows 'localhost:27017 > NoExcuseDB'. There are buttons for 'Open MongoDB shell', 'Create collection', and 'Refresh'. A table lists the collections:

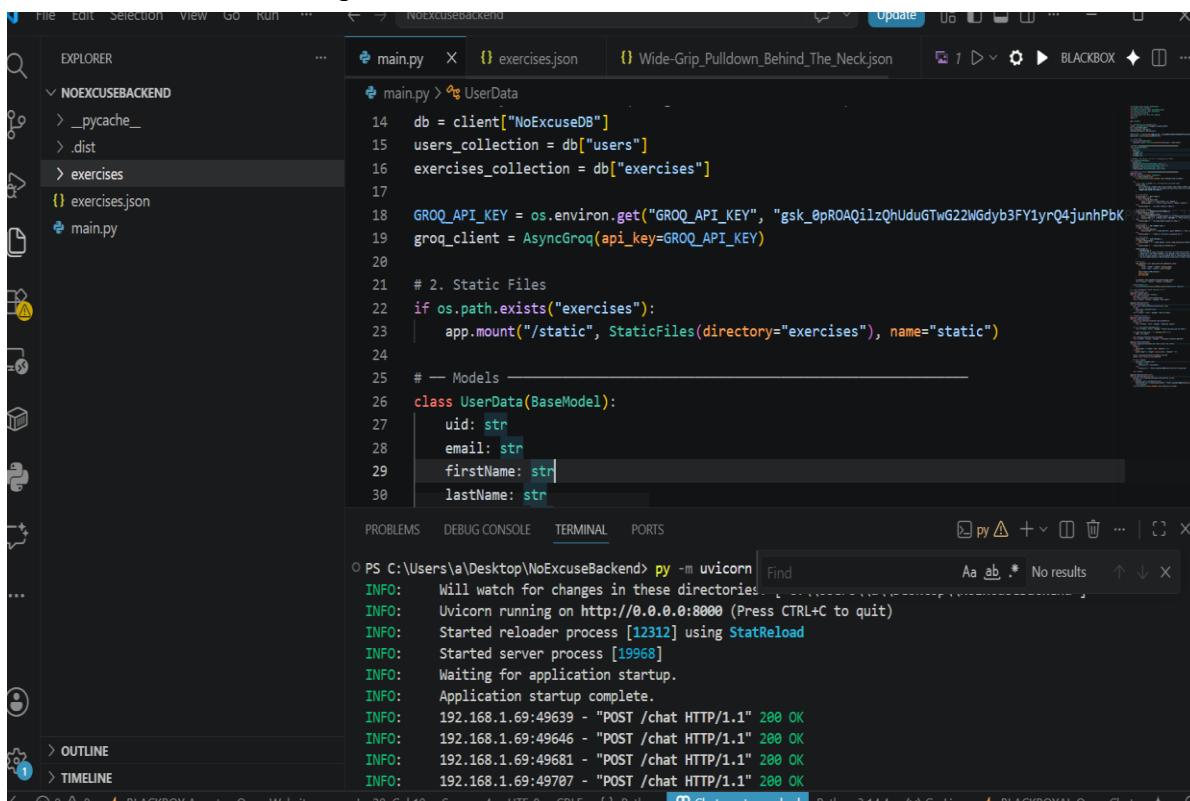
Collection name	Properties	Storage size	Data size	Documents	Avg. Document size	Indexes	Total Index size
exercises	-	372.74 kB	935.35 kB	873	1.07 kB	1	24.58 kB
users	-	24.58 kB	159.00 B	1	159.00 B	1	24.58 kB

FIGURE A.8 : Affichage Logcat de la charge JSON envoyée au backend

La figure A.8 affiche la capture des journaux de débogage d'Android Studio. On y constate la bonne compilation en tâche d'arrière-plan du contexte JSON extrait de SQLite.

FIGURE A.9 : Logs d'exécution du serveur d'API REST FastAPI

La figure A.9 montre le terminal de démarrage et de traitement des requêtes de FastAPI, affichant la réception de la méthode POST issue de Retrofit.



The screenshot shows the VS Code editor with the 'main.py' file open. The code defines a FastAPI application with a 'UserData' model and a 'POST /chat' endpoint. The terminal at the bottom shows the server startup logs and several successful POST requests.

```

14 db = client["NoExcuseDB"]
15 users_collection = db["users"]
16 exercises_collection = db["exercises"]
17
18 GROQ_API_KEY = os.environ.get("GROQ_API_KEY", "gsk_0pROAQilzQhUduGTW622WGdyb3FY1yrQ4junhPbK
19 groq_client = AsyncGroq(api_key=GROQ_API_KEY)
20
21 # 2. Static Files
22 if os.path.exists("exercises"):
23     app.mount("/static", StaticFiles(directory="exercises"), name="static")
24
25 # -- Models
26 class UserData(BaseModel):
27     uid: str
28     email: str
29     firstName: str
30     lastName: str

```

```

PS C:\Users\A\Desktop\NoExcuseBackend> py -m uvicorn
INFO: Will watch for changes in these directories:
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [12312] using StatReload
INFO: Started server process [19968]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 192.168.1.69:49639 - "POST /chat HTTP/1.1" 200 OK
INFO: 192.168.1.69:49646 - "POST /chat HTTP/1.1" 200 OK
INFO: 192.168.1.69:49681 - "POST /chat HTTP/1.1" 200 OK
INFO: 192.168.1.69:49707 - "POST /chat HTTP/1.1" 200 OK

```

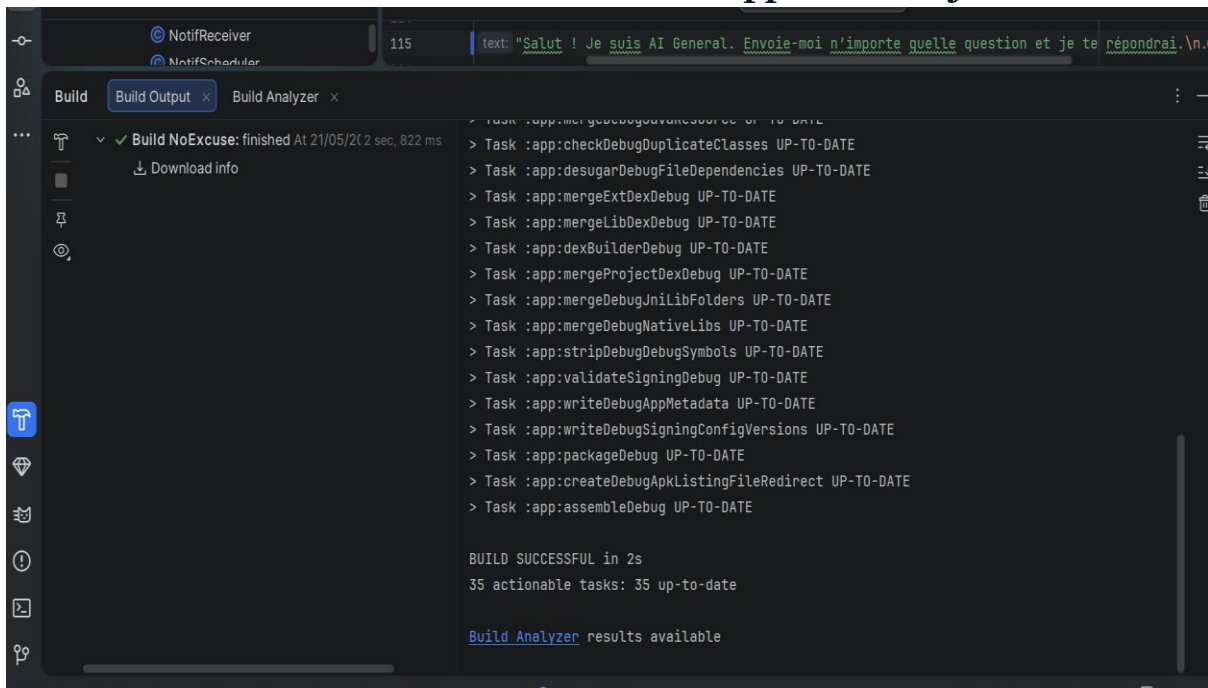


FIGURE A.10 : Rapport de build réussi de la console Gradle

La figure A.10 capture la validation finale du compilateur Gradle indiquant le succès du processus de packaging de l'APK d'installation.